Below is a **real-world, day-to-day used list of Spring Boot classes + methods** (across Spring Boot versions, practically from Boot **1.x → 3.x**) that developers commonly use in production projects.

⚡ NOTE: Spring Boot itself is mainly an "auto-configuration + starter + conventions" layer. Most *day-to-day coding* is done via **Spring Framework + Spring Data + Spring Security + Boot utilities**.
So the table includes **Boot + Spring ecosystem classes that are used in real applications daily**.

✅ Spring Boot + Spring Ecosystem: Most Common Classes & Methods (Day-to-Day Use)

1) Boot Application Startup / Core Boot

Class	Common Methods	Use Case in Real Apps	Example
SpringApplication	run(), setBannerMode(), setAdditionalProfiles()	Start Boot app, profile-based startup	SpringApplication.run(App.class,args)
SpringBootApplication	*(annotation)*	Main entry point: enables AutoConfig + ComponentScan	@SpringBootApplication public class App{}
ApplicationRunner	run(ApplicationArguments)	Run code after app startup (load cache, seed DB)	preload default admin user
CommandLineRunner	run(String...)	Similar to runner, CLI args support	seed data
ApplicationArguments	getOptionNames(), getOptionValues()	Read app startup args	--env=dev
SpringBootVersion	getVersion()	logging/debugging version at runtime	print boot version
Banner	printBanner()	Custom Boot banner	custom startup banner
SpringApplicationBuilder	profiles(), sources(),	advanced	multi-module

	run()	bootstrapping	apps

---

2) Dependency Injection / Bean Management

Class	Common Methods	Use Case	Example
ApplicationContext	getBean(), containsBean(), getEnvironment()	access beans dynamically	load dynamic strategy
BeanFactory	getBean()	low-level DI	framework integration
@Component	*(annotation)*	marks a bean	service/helper beans
@Service	*(annotation)*	service layer	business logic
@Repository	*(annotation)*	DAO layer with exception translation	database operations
@Configuration	*(annotation)*	define beans manually	external clients
@Bean	*(annotation method)*	create bean programmatically	RestTemplate, WebClient
@Autowired	*(annotation)*	inject dependencies	constructor injection
@Qualifier	*(annotation)*	choose specific bean	@Qualifier("primaryMailer")
@Primary	*(annotation)*	default bean selection	multi-impl beans
@Lazy	*(annotation)*	lazy bean creation	optimize startup
@Value	*(annotation)*	inject property value	@Value("${app.name}")
Environment	getProperty(), acceptsProfiles()	read config at runtime	multi-env logic

3) Configuration Properties / Application.yml

Class	Methods	Use Case	Example
@ConfigurationProperties	*(annotation)*	bind YAML configs into POJO	database/email settings
Binder	bind(), bindOrCreate()	bind configs manually	dynamic property loading
@EnableConfigurationProperties	*(annotation)*	enable properties binding	for custom starter

4) REST APIs (Spring MVC / Web)

Class	Methods	Use Case	Example
@RestController	*(annotation)*	REST endpoints	/api/users
@Controller	*(annotation)*	MVC page controllers	thymeleaf apps
@RequestMapping	*(annotation)*	base mapping	/api/v1
@GetMapping	*(annotation)*	GET endpoint	fetch record
@PostMapping	*(annotation)*	POST endpoint	create record
@PutMapping	*(annotation)*	PUT endpoint	update record
@DeleteMapping	*(annotation)*	delete endpoint	delete record
@PathVariable	*(annotation)*	extract path param	/users/{id}
@RequestParam	*(annotation)*	query param	/users?page=1
@RequestBody	*(annotation)*	JSON payload to object	create request
@RequestHeader	*(annotation)*	header reading	auth token
ResponseEntity	ok(), status(),	custom HTTP	return 201 + body

	badRequest(), body()	response	
HttpStatus	enums	status codes	404, 500
@ResponseStatus	*(annotation)*	return fixed status	201 CREATED
@CrossOrigin	*(annotation)*	CORS settings	frontend integration
@Valid	*(annotation)*	validate request object	input validation
BindingResult	hasErrors(), getAllErrors()	validation errors	form validation
ServletUriComponentsBuilder	fromCurrentRequest(), build()	generate location URI	after create API
WebMvcConfigurer	addCorsMappings(), addInterceptors()	global MVC config	cors, interceptors

---

5) Exception Handling + API Error Design

Class	**Methods**	**Use Case**	**Example**
@ControllerAdvice	*(annotation)*	global exception handler	common API errors
@ExceptionHandler	*(annotation)*	catch specific exception	handle UserNotFoundException
ResponseStatusException	constructor	throw HTTP errors	throw new ResponseStatusException(...)
ProblemDetail *(Boot 3+)*	forStatus(), setTitle(), setDetail()	modern RFC 7807 error response	standardized errors

---

6) Validation (Jakarta Validation / Hibernate Validator)

Annotation/Class	**Methods**	**Use Case**	**Example**

@NotNull	-	mandatory	name required
@NotBlank	-	non-empty string	email required
@Size	-	length range	password rules
@Min, @Max	-	numeric limits	Age
@Email	-	email format	login email
Validator	validate()	manual validation	service-level validation

---

7) Database + JPA (Most Used in Companies)

Class	**Methods**	**Use Case**	**Example**
JpaRepository<T,ID>	save(), findById(), findAll(), deleteById()	standard CRUD	user management
CrudRepository	save(), findAll()	minimal CRUD	Microservices
PagingAndSortingRepository	findAll(Pageable)	pagination	listing endpoints
Page	getContent(), getTotalElements()	page response	REST pagination
Pageable	ofSize(), withPage()	pagination input	requests
PageRequest	of(page,size,sort)	create pageable	controller
Sort	by(), descending()	sorting	/users?sort=name
@Entity	*(annotation)*	JPA entity	DB mapping
@Id	*(annotation)*	primary key	id
@GeneratedValue	*(annotation)*	auto id	identity/seq
@OneToMany	*(annotation)*	relations	user-orders
@ManyToOne	*(annotation)*	relations	order-user
@JoinColumn	*(annotation)*	join column	foreign keys
@Query	*(annotation)*	custom	select u from User

		JPQL/SQL	u
EntityManager	persist(), merge(), createQuery()	complex queries	custom DAO

---

8) Transactions

Class/Annotation	Methods	Use Case	Example
@Transactional	*(annotation)*	atomic DB operations	payment + order
TransactionTemplate	execute()	programmatic transaction	conditional tx control

---

9) Spring Security (Used Daily)

Class	Methods	Use Case	Example
SecurityFilterChain	config DSL	secure endpoints	permit/deny routes
HttpSecurity	authorizeHttpRequests(), csrf(), sessionManagement()	define security rules	JWT stateless
AuthenticationManager	authenticate()	login auth	/login
PasswordEncoder	encode(), matches()	password hashing	BCrypt
BCryptPasswordEncoder	encode(), matches()	most used encoder	store password
UserDetailsService	loadUserByUsernameO	load user from DB	login
GrantedAuthority	getAuthority()	roles/permissions	ROLE_ADMIN
@EnableMethodSecurity	*(annotation)*	enable @PreAuthorize	RBAC
@PreAuthorize	SpEL	method-level access	admin-only methods

---

10) HTTP Clients (Calling External APIs)

Class	Methods	Use Case	Example
RestTemplate	getForObject(), postForEntity(), exchange()	legacy HTTP calls	call payment gateway
WebClient	get(), post(), retrieve(), bodyToMono()	modern reactive client	microservice-to-microservice
HttpHeaders	set(), add()	custom headers	auth token
HttpEntity	constructors	request entity	headers+body

---

11) Logging (Practical)

Class	Methods	Use Case	Example
Logger (SLF4J)	info(), debug(), error()	app logs	request + error logs
LoggerFactory	getLogger()	create logger	private static final Logger log=...
MDC	put(), get(), clear()	traceId / correlationId logs	distributed tracing

---

12) Scheduling + Async

Class/Annotation	Methods	Use Case	Example
@EnableScheduling	-	enable scheduler	cron jobs
@Scheduled	-	cron / fixed delay jobs	nightly reports
@EnableAsync	-	async methods	email sending

@Async	-	async method execution	background tasks
TaskExecutor	execute()	thread pool	high load async
ThreadPoolTaskExecutor	setters	configure executor	custom pool

---

13) Caching

Class/Annotation	Methods	Use Case	Example
@EnableCaching	-	enable cache	Redis cache
@Cacheable	-	cache results	product list
@CachePut	-	update cache	update product
@CacheEvict	-	remove cache	delete product

---

14) Testing (Most Used in Projects)

Class/Annotation	Methods	Use Case	Example
@SpringBootTest	-	integration testing	load full context
@WebMvcTest	-	controller testing	test REST APIs
MockMvc	perform(), andExpect()	API unit tests	validate status+json
@MockBean	-	mock Spring bean	mock services
TestRestTemplate	getForEntity(), postForEntity()	API integration tests	full stack tests
@DataJpaTest	-	repository tests	DB layer testing

✅ **Real-Time Example (End-to-End Use)**

 Use case: Create User REST API (Controller + Service + Repository + Validation)

```
@RestController
@RequestMapping("/api/users")
public class UserController {

   private final UserService service;

   public UserController(UserService service) {
      this.service = service;
   }

   @PostMapping
   public ResponseEntity<User> create(@Valid @RequestBody CreateUserRequest req) {
      User saved = service.create(req);
      return ResponseEntity.status(HttpStatus.CREATED).body(saved);
   }

   @GetMapping("/{id}")
   public ResponseEntity<User> find(@PathVariable Long id) {
      return ResponseEntity.ok(service.findById(id));
   }
}
@Service
public class UserService {

   private final UserRepository repo;
   private final PasswordEncoder encoder;

   public UserService(UserRepository repo, PasswordEncoder encoder) {
      this.repo = repo;
      this.encoder = encoder;
   }

   @Transactional
   public User create(CreateUserRequest req) {
      User u = new User();
      u.setEmail(req.getEmail());
      u.setPassword(encoder.encode(req.getPassword()));
      return repo.save(u);
   }

   public User findById(Long id) {
      return repo.findById(id)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "User not
found"));
   }
}
```

```
public interface UserRepository extends JpaRepository<User, Long> {
   Optional<User> findByEmail(String email);
}
```

If you want, I can also create a **master table of 250+ classes/methods** split into modules:

✅ Spring Boot Core
✅ Spring MVC
✅ Spring Data JPA
✅ Spring Security
✅ Spring Batch
✅ Spring Cloud (Eureka, Gateway, Config, OpenFeign)
✅ Kafka / RabbitMQ
✅ Actuator
✅ Micrometer / Observability (Boot 3+)

…and provide it as a **single mega table**.

Below is the **MASTER TABLE (250+ commonly used Spring Boot + Spring ecosystem classes & methods)** split module-wise (Boot 1.x → Boot 3.x concepts included).

⭑ This is focused on **day-to-day practical development**, not rare internal framework classes.

✅ MASTER TABLE (250+ Spring Boot Classes/Methods) Split by Modules

---

✅ A) Spring Boot Core + Startup + Auto Configuration (30+)

Module	Class / Annotation	Common Methods	Real Use Case
Boot Core	@SpringBootApplication	—	Main entry enabling Boot features
Boot Core	SpringApplication	run(), setBannerMode(), setAddCommandLineProperties()	Start app, customize startup
Boot Core	SpringApplicationBuilder	profiles(), sources(), run()	Multi-profile startup
Boot Core	ApplicationRunner	run()	Run post-start tasks
Boot Core	CommandLineRunner	run()	Startup initialization
Boot Core	ApplicationArguments	getSourceArgs(),	Read startup args

		getOptionNames()	
Boot Core	@EnableAutoConfiguration	—	Auto configure beans
Boot Core	@ComponentScan	—	Scan beans
Boot Core	@Configuration	—	Manual config class
Boot Core	@Bean	—	Create explicit bean
Boot Core	@Import	—	Import configs
Boot Core	@Profile	—	Beans per environment
Boot Core	Environment	getProperty(), getActiveProfiles()	Profile-based config
Boot Core	ConfigurableEnvironment	setActiveProfiles()	Activate profile programmatically
Boot Core	@ConditionalOnProperty	—	Enable feature flag
Boot Core	@ConditionalOnClass	—	Auto config if class exists
Boot Core	@ConditionalOnMissingBean	—	Override default bean
Boot Core	@ConditionalOnBean	—	Enable only if bean present
Boot Core	SpringBootVersion	getVersion()	Print version
Boot Core	@ConfigurationProperties	—	Strong typed properties
Boot Core	@EnableConfigurationProperties	—	Register config props
Boot Core	Binder	bind(), bindOrCreate()	Bind properties programmatically
Boot Core	ApplicationContext	getBean(), getBeansOfType()	Dynamic bean usage
Boot Core	ConfigurableApplicationContext	close(), publishEvent()	Context lifecycle
Boot Core	ApplicationEventPublisher	publishEvent()	Publish custom events

Boot Core	@EventListener	—	Listen events
Boot Core	ApplicationListener	onApplicationEvent()	Low-level event listener
Boot Core	@Order	—	Priority in bean execution
Boot Core	SmartLifecycle	start(), stop()	Control background services

---

✅ B) Dependency Injection + Bean Lifecycle (25+)

Module	Class / Annotation	Methods	Real Use Case
DI	@Component	—	Bean
DI	@Service	—	Service layer
DI	@Repository	—	DAO layer
DI	@Controller	—	MVC controller
DI	@RestController	—	REST controller
DI	@Autowired	—	Inject dependencies
DI	@Qualifier	—	Choose implementation
DI	@Primary	—	Default bean
DI	ObjectProvider<T>	getIfAvailable(), getObject()	Optional injection
DI	BeanFactory	getBean()	Low-level DI
DI	BeanDefinitionRegistry	registerBeanDefinition()	Dynamic bean registration
DI	@Value	—	inject property
DI	@Lazy	—	Lazy init
DI	@Scope	—	prototype/singleton/request
DI	@PostConstruct	—	init method

DI	@PreDestroy	—	Cleanup
DI	InitializingBean	afterPropertiesSet()	init interface
DI	DisposableBean	destroy()	cleanup interface
DI	BeanNameAware	setBeanName()	bean meta
DI	EnvironmentAware	setEnvironment()	env aware bean
DI	ApplicationContextAware	setApplicationContext()	access ctx
DI	BeanPostProcessor	postProcessBeforeInitialization()	modify beans
DI	FactoryBean	getObject(), getObjectType()	factory beans
DI	@DependsOn	—	enforce init order
DI	@ImportResource	—	load XML configs

✅ C) Spring MVC / REST API Development (40+)

Module	Class / Annotation	Common Methods	Use Case
Web MVC	@RequestMapping	—	map endpoint
Web MVC	@GetMapping	—	GET
Web MVC	@PostMapping	—	POST
Web MVC	@PutMapping	—	PUT
Web MVC	@PatchMapping	—	PATCH
Web MVC	@DeleteMapping	—	DELETE
Web MVC	@PathVariable	—	path param
Web MVC	@RequestParam	—	query param
Web MVC	@RequestBody	—	JSON to object
Web MVC	@RequestHeader	—	header read
Web MVC	@CookieValue	—	cookie read

Web MVC	@ModelAttribute	—	bind form data
Web MVC	ResponseEntity	ok(), status(), body()	HTTP response
Web MVC	HttpStatus	enums	status codes
Web MVC	HttpHeaders	add(), set()	Headers
Web MVC	HttpEntity<T>	constructors	header+body
Web MVC	ServletUriComponentsBuilder	fromCurrentRequest()	build location header
Web MVC	@ResponseStatus	—	fixed status
Web MVC	@CrossOrigin	—	enable CORS
Web MVC	HandlerInterceptor	preHandle(), postHandle()	auth/logging
Web MVC	WebMvcConfigurer	addCorsMappings(), addInterceptors()	global config
Web MVC	CorsRegistry	addMapping()	cors rules
Web MVC	LocaleResolver	setLocale()	i18n
Web MVC	MultipartFile	getBytes(), transferTo()	file upload
Web MVC	@RequestPart	—	multipart request
Web MVC	@SessionAttributes	—	session state
Web MVC	RedirectAttributes	addFlashAttribute()	redirect data
Web MVC	Model	addAttribute()	mvc view model
Web MVC	ModelAndView	addObject(), setViewName()	pages
Web MVC	@InitBinder	—	binder config
Web MVC	WebDataBinder	registerCustomEditor()	custom parsing
Web MVC	@ControllerAdvice	—	global exception
Web MVC	@ExceptionHandler	—	exception mapping

Web MVC	ResponseStatusException	constructors	throw status errors
Web MVC	ProblemDetail (Boot 3)	forStatus(), setters	RFC7807 errors
Web MVC	ResponseBodyAdvice	beforeBodyWrite()	response wrapper
Web MVC	@RestControllerAdvice	—	rest exception handler
Web MVC	@Validated	—	bean validation
Web MVC	BindingResult	hasErrors(), getAllErrors()	validation errors
Web MVC	@DateTimeFormat	—	date parsing

✅ D) Validation (Jakarta / Hibernate Validator) (20+)

Module	Annotation/Class	Methods	Use Case
Validation	@Valid	—	validate request
Validation	Validator	validate()	manual validation
Validation	@NotNull	—	required
Validation	@NotBlank	—	string required
Validation	@NotEmpty	—	list required
Validation	@Size	—	min/max
Validation	@Min, @Max	—	numeric
Validation	@Positive	—	>0
Validation	@Negative	—	<0
Validation	@Email	—	email
Validation	@Pattern	—	regex
Validation	@Future, @Past	—	date constraints
Validation	@AssertTrue	—	business rules
Validation	ConstraintViolation	getMessage(), getPropertyPath()	validation errors

Validation	MethodArgumentNotValidException	getBindingResult()	rest errors
Validation	ConstraintViolationException	getConstraintViolations()	DB/service errors
Validation	@Validated	—	group validation
Validation	ValidationException	—	general errors
Validation	@Digits	—	decimal limits
Validation	@DecimalMin/Max	—	numeric rules

✅ E) Spring Data JPA / Hibernate (45+)

Module	Class / Annotation	Common Methods	Use Case
JPA	JpaRepository<T,ID>	save(), findById(), findAll(), deleteById()	CRUD
JPA	CrudRepository	save(), existsById()	basic CRUD
JPA	PagingAndSortingRepository	findAll(Pageable)	pagination
JPA	Page	getContent(), getTotalElements()	response paging
JPA	Pageable	getPageNumber(), getPageSize()	paging input
JPA	PageRequest	of()	create pageable
JPA	Sort	by(), descending()	sorting
JPA	Specification<T>	toPredicate()	dynamic filters
JPA	JpaSpecificationExecutor	findAll(Specification)	dynamic query
JPA	@Entity	—	entity
JPA	@Table	—	table mapping
JPA	@Id	—	pk

JPA	@GeneratedValue	—	auto pk
JPA	@Column	—	column mapping
JPA	@OneToOne	—	relation
JPA	@OneToMany	—	relation
JPA	@ManyToOne	—	relation
JPA	@ManyToMany	—	relation
JPA	@JoinColumn	—	fk
JPA	@JoinTable	—	m:n relation
JPA	@Enumerated	—	enum mapping
JPA	@Embedded	—	value objects
JPA	@Embeddable	—	reusable fields
JPA	@Lob	—	large text/blob
JPA	@Version	—	optimistic locking
JPA	@Transient	—	not persisted
JPA	@Query	—	custom query
JPA	@Modifying	—	update/delete query
JPA	EntityManager	persist(), merge(), createQuery()	custom dao
JPA	TypedQuery	getResultList(), getSingleResult()	query results
JPA	@Transactional	—	transaction
JPA	TransactionTemplate	execute()	programmatic tx
JPA	@EntityListeners	—	audit hooks
JPA	@CreatedDate	—	auditing
JPA	@LastModifiedDate	—	auditing
JPA	@CreatedBy	—	auditing

JPA	@LastModifiedBy	—	auditing
JPA	AuditorAware	getCurrentAuditor()	who modified
JPA	@EnableJpaAuditing	—	enable audit
JPA	@NamedQuery	—	predefined queries
JPA	@NamedNativeQuery	—	native queries
JPA	Query	setParameter(), executeUpdate()	native/JPQL

✅ F) Spring JDBC (10+)

Module	Class	Methods	Use Case
JDBC	JdbcTemplate	query(), queryForObject(), update()	SQL queries
JDBC	NamedParameterJdbcTemplate	query(), update()	named params
JDBC	RowMapper<T>	mapRow()	row mapping
JDBC	ResultSetExtractor<T>	extractData()	large result
JDBC	DataSource	getConnection()	db connection
JDBC	SQLExceptionTranslator	translate()	error handling
JDBC	SimpleJdbcInsert	execute(), usingTableName()	inserts
JDBC	BatchPreparedStatementSetter	setValues()	batch insert
JDBC	PreparedStatementCreator	createPreparedStatement()	custom stmt
JDBC	GeneratedKeyHolder	getKey()	auto keys

✅ G) Spring Security (45+)

Module	Class/Annotation	Methods	Use Case
Security	SecurityFilterChain	config DSL	security rules
Security	HttpSecurity	authorizeHttpRequests(), csrf(), oauth2Login()	configure security
Security	AuthenticationManager	authenticate()	login
Security	Authentication	getName(), getAuthorities()	current user
Security	SecurityContextHolder	getContext(), clearContext()	auth context
Security	UserDetailsService	loadUserByUsername()	load user
Security	UserDetails	getUsername(), getPassword()	user principal
Security	GrantedAuthority	getAuthority()	roles
Security	SimpleGrantedAuthority	constructor	role mapping
Security	PasswordEncoder	encode(), matches()	hashing
Security	BCryptPasswordEncoder	methods	common hashing
Security	DaoAuthenticationProvider	setters	database auth
Security	UsernamePasswordAuthenticationToken	constructors	auth token
Security	OncePerRequestFilter	doFilterInternal()	JWT filter
Security	AuthenticationEntryPoint	commence()	401 response
Security	AccessDeniedHandler	handle()	403 response
Security	@EnableWebSecurity	—	enable security
Security	@EnableMethodSecurity	—	@PreAuthorize
Security	@PreAuthorize	—	method security
Security	@PostAuthorize	—	after execution check
Security	CsrfToken	getToken()	csrf protection

Security	SessionCreationPolicy	enum	stateless JWT
Security	LogoutHandler	logout()	logout
Security	LogoutSuccessHandler	onLogoutSuccess()	logout success
Security	OAuth2User	getAttributes()	social login
Security	OAuth2AuthenticationToken	methods	oauth principal
Security	JwtDecoder	decode()	decode JWT
Security	JwtEncoder	encode()	issue JWT
Security	CorsConfigurationSource	getCorsConfiguration()	cors
Security	AuthenticationSuccessHandler	onAuthenticationSuccess()	login success
Security	AuthenticationFailureHandler	onAuthenticationFailure()	login fail
Security	AuthorizationManager	check()	custom auth
Security	@RolesAllowed	—	role check
Security	@Secured	—	role check
Security	@AuthenticationPrincipal	—	inject user
Security	SecurityContext	getAuthentication()	auth object
Security	User	builder()	in-memory users
Security	InMemoryUserDetailsManager	createUser()	testing
Security	OpaqueTokenIntrospector	introspect()	OAuth token validation
Security	BearerTokenAuthenticationFilter	filter	oauth2 resource server
Security	AuthenticationConverter	convert()	extract auth token
Security	RequestMatcher	matches()	match endpoints
Security	AntPathRequestMatcher	constructors	URL match

Security	LogoutConfigurer	logoutUrl()	configure logout

✅ H) Scheduling + Async + Threads (15+)

Module	Class/Annotation	Methods	Use Case
Async	@EnableScheduling	—	scheduler
Async	@Scheduled	—	cron tasks
Async	@EnableAsync	—	async
Async	@Async	—	background tasks
Async	TaskExecutor	execute()	thread pool
Async	ThreadPoolTaskExecutor	setters	configure pool
Async	TaskScheduler	schedule()	schedule tasks
Async	ThreadPoolTaskScheduler	setters	pool scheduler
Async	SchedulingConfigurer	configureTasks()	dynamic tasks
Async	AsyncConfigurer	getAsyncExecutor()	executor config
Async	@AsyncResult	constructors	legacy async result
Async	CompletableFuture	supplyAsync(), thenApply()	async results
Async	Future	get()	async result
Async	ListenableFuture	callbacks	old spring async
Async	@EnableScheduling	—	scheduling jobs

✅ I) Caching (10+)

Module	Annotation/Class	Methods	Use Case
Cache	@EnableCaching	—	enable caching
Cache	@Cacheable	—	cache method output

Cache	@CachePut	—	update cache
Cache	@CacheEvict	—	evict cache
Cache	CacheManager	getCache()	manage caches
Cache	Cache	get(), put(), evict()	cache ops
Cache	ConcurrentMapCacheManager	—	default cache
Cache	RedisCacheManager	builder	redis cache
Cache	CacheResolver	resolveCaches()	dynamic cache
Cache	KeyGenerator	generate()	custom keys

✅ J) Observability / Metrics / Actuator (Boot 2+ / 3+) (20+)

Module	Class/Endpoint	Methods	Use Case
Actuator	@Endpoint	—	custom actuator endpoint
Actuator	@ReadOperation	—	GET actuator
Actuator	@WriteOperation	—	POST actuator
Actuator	HealthIndicator	health()	custom health
Actuator	Health	up(), down(), withDetail()	health response
Actuator	InfoContributor	contribute()	build info
Metrics	MeterRegistry	counter(), timer(), gauge()	metrics
Metrics	Counter	increment()	count events
Metrics	Timer	record()	response times
Metrics	Gauge	—	memory/load
Metrics	@Timed	—	timing methods
Tracing	ObservationRegistry	observe()	observation (Boot 3)

Tracing	Observation	start(), stop()	tracing spans
Tracing	Tracer	nextSpan()	distributed tracing
Logging	MDC	put(), clear()	request trace id
Actuator	HttpTraceRepository	—	http trace (deprecated older)
Actuator	MetricsEndpoint	—	actuator metrics
Actuator	HealthEndpoint	—	actuator health
Actuator	InfoEndpoint	—	info
Actuator	ShutdownEndpoint	—	shutdown

✅ K) Testing (25+)

Module	Class/Annotation	Methods	Use Case
Testing	@SpringBootTest	—	integration tests
Testing	@WebMvcTest	—	controller tests
Testing	@DataJpaTest	—	repo tests
Testing	@MockBean	—	mock beans
Testing	@SpyBean	—	partial mock
Testing	MockMvc	perform()	test api
Testing	MockMvcRequestBuilders	get(), post()	build req
Testing	ResultActions	andExpect()	assertions
Testing	jsonPath()	—	json verify.
Testing	TestRestTemplate	getForEntity(), postForEntity()	API test
Testing	@ExtendWith	—	JUnit extension
Testing	@BeforeEach	—	setup
Testing	@AfterEach	—	teardown

Testing	Assertions	assertEquals(), assertThrows()	assert
Testing	Mockito	when(), verify()	mocks
Testing	@TestConfiguration	—	test beans
Testing	@AutoConfigureMockMvc	—	enable mockmvc
Testing	@DirtiesContext	—	reset ctx
Testing	@Transactional	—	rollback tests
Testing	TestEntityManager	persist(), flush()	jpa tests
Testing	@Sql	—	SQL scripts
Testing	@ActiveProfiles	—	test profile
Testing	MockBean	—	override beans
Testing	OutputCaptureExtension	—	capture logs
Testing	@TestPropertySource	—	override props

✅ This master table already contains **250+ items** (classes + annotations + methods).

If you want, I can also provide:

✅ **Spring Cloud Master Table (Eureka, Config Server, Gateway, Feign, CircuitBreaker, Resilience4j)** (150+ items)
✅ **Kafka + RabbitMQ Spring Master Table** (100+ items)
✅ **Spring Batch Master Table** (80+ items)

Just tell me which module you want next.